Improves Neural Acoustic Word Embeddings Query by Example Spoken Term Detection with Wav2vec Pretraining and Circle Loss

Zhaoqi Li[1,2], *Long Wu*[1,2], *Ta Li*[1,2], *Yonghong Yan*[1,2,3]

[1]Key Laboratory of Speech Acoustics and Content Understanding, Institute of Acoustics, Chinese Academy of Sciences, China
[2]University of Chinese Academy of Sciences, China
[3]Xinjiang Key Laboratory of Minority Speech and Language Information Processing, Xinjiang Technical Institute of Physics and Chemistry, Chinese Academy of Sciences, China
{lizhaoqi,wulong,lita,yanyonghong}@hccl.ioa.ac.cn

2021 12th International Symposium on Chinese Spoken Language Processing (ISCSLP) | 978-1-7281-6994-1/20/$31.00 ©2021 IEEE | DOI: 10.1109/ISCSLP49672.2021.9362065

Abstract

Query by example spoken term detection (QbE-STD) is a popular keyword detection method in the absence of speech resources. It can build a keyword query system with decent performance when there are few labeled speeches and a lack of pronunciation dictionaries. In recent years, neural acoustic word embeddings (NAWEs) has become a commonly used QbE-STD method. To make the embedded features extracted by the neural network contain more accurate context information, we use wav2vec pre-training to improve the performance of the network. Compared with the Mel-frequency cepstral coefficients(MFCC) system, the average precision (AP) is relatively improved by 11.1%. We also find that the AP of the wav2vec and MFCC splicing system is better, demonstrating that wav2vec cannot contain all spectrum information. To accelerate the convergence speed of the splicing system, we use circle loss to replace the triplet loss, making the convergence about 40% epochs earlier on average. The circle loss also relatively increases AP by more than 4.9%. The AP of our best-performing system is 7.7% better than the wav2vec baseline system and 19.7% better than the MFCC baseline system.

Index Terms: Acoustic word embedding, wav2vec, Circle loss, Query by example spoken term detection

1. Introduction

QbE-STD is a task of querying the target speech database for subsequences similar to the query speech submitted by the user. Unlike the text query in the keyword search, QbE-STD requires a speech query, enabling language-independent search without a robust speech recognition system. Regardless of language and semantic information, this task searches by the similarity of speech data so that it can perform well under low resource conditions and not worry about the out-of-vocabulary problem.

The QbE-STD system mainly includes the following two steps: First, converting the queries and target speech into the same representations. Second, using them to calculate the probability of the query being same as the keyword occurred somewhere in the target speech as a sub-sequence. There are mainly three representations that have been used: spectral features [1, 2], phonetic posteriorgram [3, 4], and bottleneck features [5, 6, 7]. After feature extraction, Dynamic Time Warping (DTW) is a typical sequence similarity comparison method in QbE-STD tasks.

DTW [1] is a method of searching the path that can represent the minimum distance between two speech sequences. The path search is carried out frame by frame on a two-dimensional plane. The coordinates of points on the plane are the number of frames of the two speech sequences, and the similarity of frames in each point is used as the path cost. The frame-by-frame search makes it precise, and the way of path search makes it less affected by time-bending. DTW is still one of the most accurate similarity comparison algorithms. However, the search on the two-dimensional plane can only perform dynamic planning calculations in real-time, making the DTW method time-consuming. For the parallelism, search range and accuracy of DTW, several variants of DTW have been proposed to optimize: slope-constrained DTW [3], segmental DTW [4], weighted DTW [8], non-segmental DTW [9], subsequence DTW [10], subspace regularization DTW [11].

Recently, NAWEs have gradually become a popular QbE method. Compared to the complex similarity comparison of DTW, NAWEs transfer complexity to the step of embedding fixed-length vectors. Several studies have used the triplet loss Siamese network [2, 12, 13, 14, 15, 16, 17] as the embedding function. This method trains the neural network through word pairs data with weakly label indicating whether the words belong to the same category.

In the QbE-STD task, the multilingual or monolingual bottleneck feature [7] is a frequently used pre-training method. A distinct drawback of this method is that in the QbE-STD task, we often do not want to build an acoustic model. Because of low-quality annotations and incorrect alignments, some errors are often introduced to extract high-level features. wav2vec [18] is a recently proposed unsupervised pre-training method that can be used to improve speech tasks. wav2vec pre-training can be performed using unsupervised speech data. Even under the same amount of data, pre-training can reduce training time and improve performance. wav2vec has shown excellent performance in the speech recognition system [19, 20]. In this paper, we test the performance of wav2vec under QbE-STD tasks. Simultaneously, to accelerate training of the QbE-STD system with pre-training features and improve performance, we use circle loss instead of triple loss to train our network. Circle loss is better than triplet loss in terms of performance and speed.

2. NAWEs-QbE-STD System

As shown in Figure 1, NAWEs embed all the variable-length speech sequences into fixed-length vectors through a neural network as embeddings function. The neural network is trained by using word sequences with at least weak word labels. The network learns to map the same words to vectors close to each other and map different words to vectors that are far apart. In

this way, after embedding the speech sequences into the vectors, the distance between the embedding vectors can be compared to determine whether the segments are the same word directly. The method proposed in [2] using triplet bidirectional long short-term memory (LSTM) [21] neural network as the embedding function will be briefly introduced below and used as our baseline system.

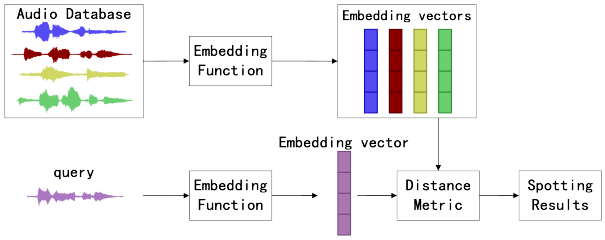

Figure 1: *NAWEs query by example spoken term detection system.*

Concretely, the system uses 3-layer BLSTM as embedding function, and stitch the last frame output vector of both direction LSTMs as embedding vector:

$$E = [hf_T; hb_1] \tag{1}$$

Where hf_T and hb_1 are the last frame output vector of the forward and backward LSTM, T is the length of the speech sequence. The neural network is trained using triplet loss function [22], defined as:

$$\mathcal{L}_{triplet} = max\{0, m + d(x_a, x_p) - d(x_a, x_n)\} \tag{2}$$

$$d(x_1, x_2) = Dis_{cosine}(x_1, x_2) = (1 - cos(x_1, x_2)) \tag{3}$$

x_a, x_p, x_n are the corresponding vectors of sequences embedded by the neural network. By minimizing triplet loss, the within-class distance $d(x_a, x_p)$ of the sample embedding vector x_a and the positive sample embedding vector x_p will decrease, while the between-class distance $d(x_a, x_n)$ of the x_a and negative sample embedding vector x_n will increase. m is a positive margin used to limit how much the between-class distance more magnificent than the within-class distance. If the between-class distance does exceed the within-class distance by more than m, triplet loss is 0. The distance we use here is the cosine distance. More details will be given in section 5.1.

3. WAV2VEC

In many application scenarios, enough labeled data and accurate dictionaries are often costly or difficult to obtain, making it hard for us to build effective speech recognition systems and keyword systems. The QbE-STD method does not need to construct a speech recognition system and performs prominently in the absence of resources. As with the QbE-STD task, the data cost of training the wav2vec model is also meager. The wav2vec feature is suitable for improving the performance of QbE-STD.

wav2vec is a pre-trained model that, through unsupervised training, enables the network to map raw speech samples to a feature space that is more representative of the characteristics of the data. Using the calculated feature vectors to replace traditional features such as MFCC can improve subsequent tasks. wav2vec model contains two convolutional neural networks, an encoder network that maps the original input audio signal to the hidden space and a contextual network that combines multiple time-step outputs of the coding network. The encoder produces a representation z_i for each time step i, and the contextual combines multiple encoder time steps into a new representation c_i for each time step i. The model is trained by minimizing the contrastive loss for each step $k = 1 \dots K$:

$$\mathcal{L}_k = -\sum(\log \sigma(\mathbf{z}_{i+k}^{\top} h_k(\mathbf{c}_i)) + \underset{\tilde{z} \sim p_n}{\lambda \mathbb{E}} [\log \sigma(-\tilde{\mathbf{z}}^{\top} h_k(\mathbf{c}_i))]) \tag{4}$$

$\tilde{\mathbf{z}}$ is the distractor that is automatically sampled from each voice during the training of the wav2vec model and conforms to the average distribution of $p_n = \frac{1}{T}$, where T is the sequence length. When calculating the expectation, the expectation is approximated by choosing ten distractors. z_{i+k} is a sample of the next k steps of $\tilde{\mathbf{z}}$. $\sigma(x) = (1/1 + exp(-x))$ is the sigmoid function, $\sigma(\mathbf{z}_{i+k}^{\top} h_k(\mathbf{c}_i))$ is the probability of $\mathbf{z}_{i+k}$ being the real sample. $h_k(\mathbf{c}_i) = W_k \mathbf{c}_i + \mathbf{b}_k$ is a step-specific affine transformation for each step, and λ is set to the number of negatives.

Finally, the wav2vec model is trained by optimizing the sum of the loss functions at each step, $\mathcal{L} = \sum_{k=1}^{K} \mathcal{L}_k$. After training, the representations produced by the context network $\mathbf{c}_i$ can be used as a traditional acoustic feature and show better performance than filter banks feature. The wav2vec model we use will be introduced in Section 5.2.

4. Circle Distance Loss

The mathematical form of circle loss proposed in [23] is as follows:

$$\begin{aligned} \mathcal{L}_{circle} = \log[1 + \sum_{j=1}^{L} \exp(\gamma(s_n^j)^2 - \gamma m^2) \\ + \sum_{i=1}^{K} \exp(\gamma(1 - s_p^i)^2 - \gamma m^2)] \end{aligned} \tag{5}$$

where s_n^j represents a between-class similarity between a negative sample and the sampled word, L is the number of negative samples, s_p^i represents a within-class similarity between a positive sample and the sampled word, and K is the number of positive samples. γ is a positive scale factor, and m is a positive margin for better similarity separation. The similarity used will be normalized between 0 and 1. The 0 means that the two vectors are entirely different, and the 1 means that the two vectors are entirely the same. The form of triplet loss frequently used in previous work is usually optimized for distance, so we slightly modified the form of circle loss:

$$\begin{aligned} \mathcal{L}'_{circle} = \log[1 + \sum_{i=1}^{K} \exp(\gamma(d_p^i)^2 - \gamma m^2) \\ + \sum_{j=1}^{L} \exp(\gamma(1 - d_n^j)^2 - \gamma m^2)] \end{aligned} \tag{6}$$

where d_p^i represents a within-class distance, and d_n^j represents a between-class distance. We use the cosine distance normalized to between 0 and 1 as the distance metric, which is:

$$d_p^i = \frac{1 - cos(x_a, x_p^i)}{2}, d_n^j = \frac{1 - cos(x_a, x_n^j)}{2} \tag{7}$$

x_a is the embedding of the sampled words in the training set, x_p^i is the embedding of the i-th positive sample, and x_n^j is the

embedding of the j-th negative sample. We sequentially select each word in the training set as the sampled word during training, then choose K positive samples and L negative samples, calculate the embedding through the BLSTM network, and finally calculate the loss and optimize the network.

The gradients of Circle loss with respect to d_p^i and d_n^j are derived as follows:

$$\frac{\partial \mathcal{L}'_{circle}}{\partial d_p^i} = Z \frac{\exp(\gamma(d_p^i)^2 - \gamma m^2)}{\sum_{l=1}^{L} \exp(\gamma(d_p^l)^2 - \gamma m^2)} \gamma(d_p^i + m) \quad (8)$$

$$\frac{\partial \mathcal{L}'_{circle}}{\partial d_n^j} = Z \frac{\exp(\gamma(d_n^j - 1)^2 - \gamma m^2)}{\sum_{k=1}^{K} \exp(\gamma(d_n^k - 1)^2 - \gamma m^2)} \gamma(d_n^j - 1 - m) \quad (9)$$

$Z = (1 - \exp(\mathcal{L}'_{circle}))$. It is easy to find that formula (8) is less than 0, and formula (9) is greater than 0. When minimizing the loss function, d_n^j will increase, and d_p^i will decrease, and larger γ will make the gradient larger.

Observing formula (8), we can know that for any within-class distance d_p^i, the gradient will be larger when the absolute value and relative value of the d_p^i are larger. Through such a gradient, the circle loss can optimize multiple within-class distances at the same time, and optimize the higher distances with greater gradients. A conclusion symmetrical of the between-class distance to formula (9) can also be obtained from formula (8). Therefore, the circle loss can optimize multiple different within-class distances and between-class distances with different gradients. The gradient will be small when a certain distance has been optimized, and the more worse distance will be optimized with a larger gradient. The circle loss has a significant advantage compare to triplet loss, which can only optimize a within-class distance and a between-class distance with the same gradient. Therefore, we used circle loss instead of triplet loss in subsequent experiments to speed up the training. The experimental results also show that circle loss also improves AP.

5. Experiment Setup

The data we use to train the embedding network is drawn from the Switchboard telephone conversational English corpus [24]. We use Kaldi [25] to extract standard Kaldi 39-dimensional MFCC+Δ + $\Delta\Delta$ in each frame. The train set contains 10959 word-segments, while the development set and test set each contains 14997 and 11951 word-segments. The entire set is the same as [13].

In our experiments, we perform the acoustic word discrimination task as an alternative to the QbE-STD task to test the approaches. Previous papers have used this task [2, 14, 15, 17]. We use all possible word-pairs, so the test set contains more than 71 million word-pairs.

5.1. Baseline System

Our baseline system structure [2] is a 3-layer BLSTM network with dropout of 0.3. All models were implemented in TensorFlow [26]. BLSTM has 256 hidden units in each direction of each layer, so the network inputs variable-length MFCC feature sequences, and finally outputs 512-dimensional fixed-length embedding vectors. We set the margin m in the triplet loss to 0.6 in all experiments and use Adam optimizer with the learning rate of 0.001 and batch size of 32.

In each training epoch, we shuffle the entire training set, randomly select one positive sample and five negative samples for each word, and then use the most indistinguishable negative samples under the current network to calculate the loss. We train for each structure for 200 epochs and select the one with the best performance on the development set as the final model. All our systems are trained using the same strategy.

While testing, two words are determined to be the same when the distance between two embedding vectors is below the threshold. We can generate the precision-recall curve by traversing all thresholds, and the area below the curve is the AP, the point of intersection with the line of the precision rate equal to the recall rate is the precision-recall break-even point (PRBEP).

5.2. wav2vec System

The wav2vec is used to optimize the system. We use the "wav2vec large" model with the best performance, which is trained using 960h Lbrispeech [27] data and published in [18], instead of training a wav2vec model by ourselves. This capacity-increasing model adds two additional linear transformations and 12 layers with increasing kernel sizes.

6. Results

6.1. wav2vec Performance

As shown in Table 1, we first train the baseline systems of MFCC and wav2vec using triplet loss. Although wav2vec does not add any additional annotation information, it obtains features containing context information through pre-training learning and achieves significantly better results than MFCC, which is the same as the performance of wav2vec on other speech tasks.

Then, to explore whether the features learned by wav2vec contain all the spectral information, which is enough to replace the low-order spectral features completely, we try to concatenate wav2vec MFCC features to train a network. To make MFCC features comparable to the influence of wav2vec, we map MFCC features to 512 dimensions through a fully connected layer and stitch them with wav2vec features to form a feature input 1024 dimensions. We train this fully connected layer and the entire network together, which will increase the number of network parameters, and because of the additional fully connected layer, the network convergence speed slows down. Furthermore, in the test, the spliced network is 15% slower than the wav2vec baseline.

Table 1: *Performance of the QbE-STD system in the Switchboard database using various features. AP and PRBEP (both higher is better) are used as evaluation metrics.*

Feature	Loss	AP(%)	PRBEP(%)
$MFCC$	$triplet$	73.82	71.12
$wav2vec$	$triplet$	82.05	77.85
$wav2vec + MFCC$	$triplet$	83.23	77.40

The result proves that the splicing feature improves AP by 1.18% compared to wav2vec, which shows that through long-term unsupervised pre-training, the learned pre-training features cannot completely cover the information of artificially extracted spectral features. However, the PRBEP of the splicing system has dropped slightly, which is because although the precision rate has been improved at most thresholds, the recall rate has dropped slightly. The addition of spectral features makes the

same words not considered the same due to substantial differences in frequency, which may be caused by the gender of the speaker. Figure 2 shows the progress of AP on the development set with the training period using various features. Although the splicing system has advantages over the pure wav2vec system, the convergence speed is significantly slower.

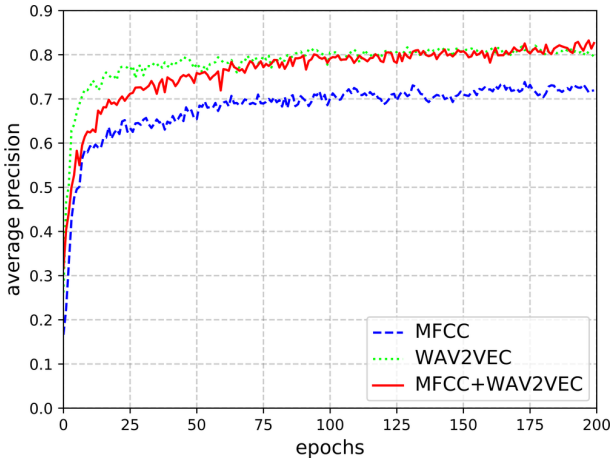

Figure 2: *Progression of the development set AP of various features with triplet loss on acoustic word discrimination.*

6.2. Circle Loss Performance

To improve the convergence speed of the splicing network, we use circle loss to replace triplet loss. We analyzed the reason why circle loss can achieve faster convergence speed in section 4. We set $\gamma = 10$ and $m = 0.6$. The other parameters and training process is the same as triplet loss.

We explored the effect of inputting different positive and negative samples in circle loss. Unlike triplet loss, which can only input one positive sample and one negative sample, circle loss allows any number of positive samples and negative samples to be input. When we use one positive sample and five negative samples to optimize the triplet loss, we can only perform five optimizations or choose the most significant loss for optimization. The circle loss can use all samples at once.

Table 2 shows the experimental results on the splice system of changing the number of negative samples. As the number of negative samples in a single calculation increases, the system performance of circle loss training improves, reaching the best at eight negative samples. Nevertheless, when the number of negative samples increases, the AP of the system becomes worse because there is only one positive sample compared to the number of negative samples, the within-class distance cannot be well trained. However, PRBEP has continued to increase. As the number of negative samples increases, the system has

Table 2: *Performance of the splice QbE-STD system using circle loss with different num of negative samples.*

Num of Negative Samples	AP(%)	PRBEP(%)
2	84.55	78.87
4	86.98	81.86
6	87.68	82.42
8	**88.33**	83.13
10	87.86	83.32
12	87.76	**83.49**

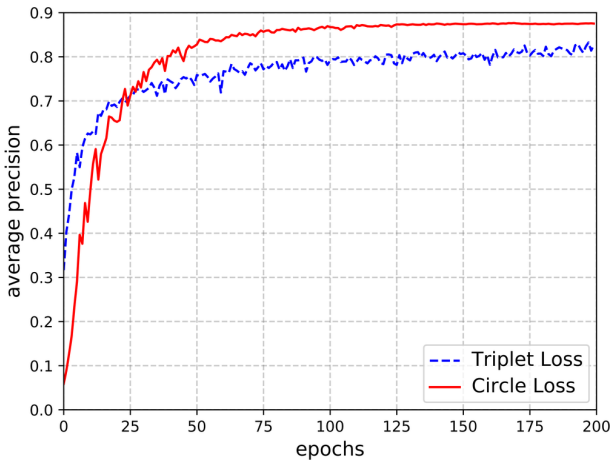

Figure 3: *Progression of the development set AP of splice system trained with circle loss and triplet loss on acoustic word discrimination.*

acquired more domain information so that the same words that have been misjudged as different words can be recalled. Therefore, the significantly increased recall rate leads to a continuous increase in PRBEP.

In the test set we extracted, many words have only two examples, so training to increase the number of positive samples cannot be performed. We tried to directly copy the training set to twice the size to enhance the number of the same words, but the experiment did not achieve the desired effect. Such operations may be conducted on other test sets in the future.

Figure 3 shows the progress of AP on the development set with the training period using circle loss and triplet loss. At the same time, we find that circle loss can also improve the performance of the system. We changed the three training systems to circle loss training with eight negative samples, and the AP relatively increased by about 5%, as shown in Table 3.

Table 3: *Performance of the QbE-STD system using circle loss with eight negative samples.*

Feature	Loss	AP(%)	PRBEP(%)
$MFCC$	$circle$	78.78	74.19
$wav2vec$	$circle$	87.15	82.87
$wav2vec + MFCC$	$circle$	88.33	83.13

7. Conclusions

In this paper, we verify the improvement of wav2vec pre-training on QbE-STD tasks. The increase of the wav2vec and MFCC features splicing system compared with the wav2vec system proves that the wav2vec pre-training feature cannot cover all the spectral information. To increase the speed of system convergence, we use circle loss to train the neural network. The convergence speed of the neural network is significantly improved, and the AP is relatively enhanced by about 5%. Then we studied the use of circle loss on the QbE task, found that a relatively advantageous training setting is to input one positive sample and eight negative samples each time. In the end, the AP of our best-performing system is relatively 7.7% better than the wav2vec baseline system and 19.7% better than the MFCC baseline system.

8. References

[1] F. Itakura, "Minimum prediction residual principle applied to speech recognition," *IEEE Transactions on acoustics, speech, and signal processing*, vol. 23, no. 1, pp. 67–72, 1975.

[2] S. Settle, K. Levin, H. Kamper, and K. Livescu, "Query-by-example search with discriminative neural acoustic word embeddings," *Proc. Interspeech 2017*, pp. 2874–2878, 2017.

[3] T. J. Hazen, W. Shen, and C. White, "Query-by-example spoken term detection using phonetic posteriorgram templates," in *2009 IEEE Workshop on Automatic Speech Recognition & Understanding*. IEEE, 2009, pp. 421–426.

[4] Y. Zhang and J. R. Glass, "Unsupervised spoken keyword spotting via segmental dtw on gaussian posteriorgrams," in *2009 IEEE Workshop on Automatic Speech Recognition & Understanding*. IEEE, 2009, pp. 398–403.

[5] H. Chen, C.-C. Leung, L. Xie, B. Ma, and H. Li, "Unsupervised bottleneck features for low-resource query-by-example spoken term detection." in *INTERSPEECH*, 2016, pp. 923–927.

[6] Y. Yuan, C.-C. Leung, L. Xie, H. Chen, B. Ma, and H. Li, "Pairwise learning using multi-lingual bottleneck features for low-resource query-by-example spoken term detection," in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2017, pp. 5645–5649.

[7] D. Ram, L. Miculicich, and H. Bourlard, "Multilingual bottleneck features for query by example spoken term detection," in *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2019, pp. 621–628.

[8] K. Levin, A. Jansen, and B. Van Durme, "Segmental acoustic indexing for zero resource keyword search," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, pp. 5828–5832.

[9] Y.-A. Chung, C.-C. Wu, C.-H. Shen, H.-Y. Lee, and L.-S. Lee, "Audio word2vec: Unsupervised learning of audio segment representations using sequence-to-sequence autoencoder," *Interspeech 2016*, pp. 765–769, 2016.

[10] M. Müller, "Dynamic time warping," *Information retrieval for music and motion*, pp. 69–84, 2007.

[11] D. Ram, A. Asaei, and H. Bourlard, "Sparse subspace modeling for query by example spoken term detection," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 6, pp. 1130–1143, 2018.

[12] H. Kamper, W. Wang, and K. Livescu, "Deep convolutional acoustic word embeddings using word-pair side information," in *2016 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2016, pp. 4950–4954.

[13] S. Settle and K. Livescu, "Discriminative acoustic word embeddings: Recurrent neural network-based approaches," in *2016 IEEE Spoken Language Technology Workshop (SLT)*. IEEE, 2016, pp. 503–510.

[14] W. He, W. Wang, and K. Livescu, "Multi-view recurrent neural acoustic word embeddings," *Proc. ICLR*, 2017.

[15] M. Jung, H. Lim, J. Goo, Y. Jung, and H. Kim, "Additional shared decoder on siamese multi-view encoders for learning acoustic word embeddings," in *2019 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2019, pp. 629–636.

[16] K. Audhkhasi, A. Rosenberg, A. Sethy, B. Ramabhadran, and B. Kingsbury, "End-to-end asr-free keyword search from speech," *IEEE Journal of Selected Topics in Signal Processing*, vol. 11, no. 8, pp. 1351–1359, 2017.

[17] H. Kamper, K. Livescu, and S. Goldwater, "An embedded segmental k-means model for unsupervised segmentation and clustering of speech," in *2017 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*. IEEE, 2017, pp. 719–726.

[18] S. Schneider, A. Baevski, R. Collobert, and M. Auli, "wav2vec: Unsupervised pre-training for speech recognition," *Proc. Interspeech 2019*, pp. 3465–3469, 2019.

[19] A. Baevski and A. Mohamed, "Effectiveness of self-supervised pre-training for asr," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7694–7698.

[20] M. Rivière, A. Joulin, P.-E. Mazaré, and E. Dupoux, "Unsupervised pretraining transfers well across languages," in *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 7414–7418.

[21] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[22] E. Hoffer and N. Ailon, "Deep metric learning using triplet network," in *International Workshop on Similarity-Based Pattern Recognition*. Springer, 2015, pp. 84–92.

[23] Y. Sun, C. Cheng, Y. Zhang, C. Zhang, L. Zheng, Z. Wang, and Y. Wei, "Circle loss: A unified perspective of pair similarity optimization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 6398–6407.

[24] J. J. Godfrey, E. C. Holliman, and J. McDaniel, "Switchboard: Telephone speech corpus for research and development," in *Acoustics, Speech, and Signal Processing, IEEE International Conference on*, vol. 1. IEEE Computer Society, 1992, pp. 517–520.

[25] D. Povey, A. Ghoshal, G. Boulianne, L. Burget, O. Glembek, N. Goel, M. Hannemann, P. Motlicek, Y. Qian, P. Schwarz *et al.*, "The kaldi speech recognition toolkit," in *IEEE 2011 workshop on automatic speech recognition and understanding*, no. CONF. IEEE Signal Processing Society, 2011.

[26] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin *et al.*, "Tensorflow: Large-scale machine learning on heterogeneous distributed systems," *arXiv preprint arXiv:1603.04467*, 2016.

[27] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, "Librispeech: an asr corpus based on public domain audio books," in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, pp. 5206–5210.